A model, a menu, and a loop

How Claude Code works

Florian Gärtner

Where we are today

	What this course is for
	Getting it running
	Try it out
	The terminal, and how to read what Claude does
	Lunch
	A working day with Claude Code
	What the model reads, predicts and weighs
→	**A model, a menu, and a loop** ← you are here
	It doesn’t remember. It’s handed everything again.
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Explain** why Claude takes several steps on its own before it answers.
2. **Connect** an MCP server to add tools to Claude’s menu.
3. **Judge** whether a rewind can take back something Claude did.

You asked for one thing

You type: *"`analysis.R` stops with an error. Fix it."*

```
→ run analysis.R                    ← error: object 'income_log' not found
→ read analysis.R                   ← [the file's text]
→ search for "income_log"           ← defined in clean_data.R, line 40
→ read clean_data.R                 ← [the file's text]
→ edit analysis.R                   ← done
→ run analysis.R                    ← runs to the end
"Fixed: analysis.R used income_log before sourcing clean_data.R."
```

Nobody told Claude to search, or to run the script again. Who decided?

Each line is a request

The model behind Claude Code only writes text and cannot open a file or run anything.

"Without tools, Claude can only respond with text." Anthropic, Claude Code documentation.

Some of that text is a request. Each `Read(…)`, `Update(…)` or `Bash(…)` line on your screen is one:

1. The model **writes a request**, in a fixed format.
2. **Claude Code**, the program around the model, **carries it out**.
3. Claude Code **pastes the result back** into the model's input, as text.

This is a **tool call**. The program that does steps 2 and 3 is the **harness**.

The harness is a program of its own

The model runs on a server. Claude Code runs on your machine, gives the model its menu and carries out its requests.

Other programs do the same job:

- **The chat window** on claude.ai is a harness too, run by Anthropic, with a menu that does not reach your disk.
- **Codex CLI** (OpenAI) and **Gemini CLI** (Google) do it for their makers' models.
- **OpenCode**, **Pi** (pi.dev) and **goose** serve models from many makers. goose also comes as a desktop app; the HRZ publishes a setup for OpenCode.

The model and the harness are separate programs. Almost any model runs in almost any harness, so you do not need Claude Code to run Claude.

The menu

Kind	What the model can ask for
Files	read, edit, create
Search	find files by name, search inside them
Execution	run a command in the terminal
Web	search the web, fetch a page

Plus tools for asking you a question or handing a task to a helper.

Then it goes round again

The model

1. reads everything so far
2. writes a request, *or* an answer

Claude Code

3. carries out the request
4. pastes the result back in

Back to 1. It stops when the model writes an answer instead of a request.

> **That is the whole “agent”**
>
> A model, a menu, and a loop.

What “agentic” means

You will meet both words everywhere: agentic AI, agentic coding, AI agents.

An **agent** is the loop you have just seen. The model picks its next step from the result of the one before, with the tools on its menu.

So who decided to search, in the opening example? The model did, after the error named `income_log`.

Agentic is looser. It comes in degrees, and it also covers fixed sequences a programmer wrote in advance, where the model only fills in each step.

Anthropic’s own documentation calls Claude Code an **agentic harness**, “the layer around the model that provides the tools and manages the context the model sees.”

An agent is a model, a menu, and a loop.

The loop is written down

Every request and every result is saved as plain text under `~/.claude/projects/`, session by session. One of my sessions:

In the file	How many
Messages I typed	5
Requests the model wrote	160
Results pasted back	160

These files hold every file and command output Claude read, unencrypted. Claude Code deletes them after 30 days by default.

Too dense to read yourself. Ask Claude to read one back when you need it.

What a rewind covers

A file can change through two items on the menu: the **file-editing tool**, or a **command** the model asked to run.

Before the file-editing tool changes a file, Claude Code keeps a copy. **Those edits can be rewound.** On screen, they are the `Write(…)` and `Update(…)` lines, and a command is a `Bash(…)` or `PowerShell(…)` line.

> ⚠ **A rewind covers the editing tool only**
>
> It does not bring back a file a command deleted or moved. Nor an email Claude sent, a database it changed or a commit it pushed.

The menu can be extended

The built-in menu covers files, the terminal and the web. You can add to it.

An **MCP server** is a small program that offers Claude Code more tools, for example to search a literature database, read your calendar or update an issue tracker. Once connected, its tools appear on the menu next to the built-in ones.

The loop stays the same, with a longer menu.

Connect an MCP server, once

You connect an MCP server once. From then on, its tools are on the menu in every session in that folder.

When you want another server, ask Claude to connect it. For OpenAlex, the open literature database, Claude runs:

```
claude mcp add --transport stdio openalex -- npx -y openalex-research-mcp
```

In a session, `/mcp` shows the connected servers and switches them on or off. Your kit already connected OpenAlex, for all your folders.

> ⚠ **Connect only MCP servers you trust**
>
> An MCP server that reads web pages can bring in instructions you did not write.

What to take away

1. The model writes text. **Claude Code turns some of that text into actions.**
2. Claude works in a loop of requests and results until it answers.
3. The menu decides what Claude can do, and **MCP servers** add to it.
4. Only edits made with the file-editing tool can be rewound.

Watch the loop on your own project

Your project · free time

- Ask Claude for something that takes several steps, such as *"Explain how the scripts in this folder fit together."*
- Watch the lines on screen: which tool, which file and which command. `Ctrl+O` shows each in full.
- Type `/mcp`. Is OpenAlex connected? If not, ask Claude to connect it. Then ask for a few recent papers on your topic, and watch which tool Claude calls.